



Technical and product documentation

Turn research into trusted PR content.

Version	1.4 (29 September 2026)
Author	Tasveeb Yousaf, BSc Computer Science, University of Debrecen
Context	DEIK.AI Challenge 2026 · AI Demo Competition · Category 2.C
Live app	citeflow-ai.streamlit.app
Source code	github.com/tasveebyousaf/citeflow

Living document: every change to CiteFlow is recorded in the Development log (section 14) and Document history.

Document history

Version	Date	Changes
1.0	25 Sep 2026	Initial build of CiteFlow (first as VeriPress) for the DEIK.AI Challenge submission. Documented retrospectively in v1.1.
1.1	27 Sep 2026	Security review and hardening (10 findings, all addressed), synced missing performance/memory fixes, privacy note and account deletion, pinned dependencies, Dependabot. This documentation created.
1.2	29 Sep 2026	PostgreSQL database backend (store_pg.py) selected by DATABASE_URL, with the JSON files kept as fallback; migration script; 44 additional database tests. Sections 4, 7, 8, 11, 12, 13 and 14 updated.
1.3	29 Sep 2026	Production-readiness work: automated test suite and GitHub checks, accuracy evaluation set, independent checker model, monitoring and Admin page, OCR for scanned PDFs, data retention, code restructured into modules, Hungarian interface, accessibility pass. New sections 12 and 15; most sections updated.
1.4	29 Sep 2026	Second evaluation run and what the two runs can and cannot show; evaluate.py now waits and retries when Google's service is busy instead of crashing. Sections 12 and 14 updated.

How this document is maintained. The PDF is generated from a script, so it is rebuilt after every change to the app. Each change adds a row to the table above and an entry to the Development log (section 14). The PDF is kept in the repository under docs/.

Contents

1. Introduction	5
1.1 Purpose of CiteFlow	5
1.2 Background and problem	5
1.3 Context: DEIK.AI Challenge 2026	5
1.4 Goals and scope	5
1.5 Intended audience of this document	5
2. Principles and rules	7
2.1 Product principles	7
2.2 Content-generation rules	7
2.3 Verification rules	7
3. User guide	9
3.1 Signing in	9
3.2 Pages	9
3.3 Creating content (Studio)	9
3.4 Deciding on flagged sentences	10
3.5 See the proof	11

3.6 Other tools	11
4. Architecture	12
4.1 Overview	12
4.2 Modules	12
4.3 Technology stack	13
4.4 Session and data flow	14
5. AI pipeline	15
5.1 Reading the paper	15
5.2 Generation	15
5.3 Verification	15
5.4 Independent checker	15
5.5 Model selection and resilience	16
5.6 Refinement, planning and stress test	16
5.7 Prompt-injection protection	16
6. Media generation	17
6.1 Designed posts	17
6.2 Carousels	17
6.3 Explainer video	17
7. Data and storage	18
7.1 Two interchangeable backends	18
7.2 Database schema	18
7.3 How files are handled	18
7.4 Consistency and concurrency	18
7.5 Hosted database and limits	19
8. Security	20
8.1 Approach	20
8.2 Threat model	20
8.3 Controls implemented	20
8.4 Review findings and status	22
8.5 Residual risks	22
9. Privacy and responsible AI	23
9.1 Personal data processed	23
9.2 Third-party processors	23
9.3 User rights and controls	23
9.4 Responsible AI	23

10. Performance	24
11. Deployment and operations	25
11.1 Run locally	25
11.2 Configuration (secrets)	25
11.3 Publish updates to GitHub	25
11.4 Streamlit Community Cloud	25
11.5 Connecting a PostgreSQL database	25
11.6 Creating team accounts	25
11.7 Incident response: leaked API key	26
12. Testing and quality assurance	27
12.1 Automated tests	27
12.2 Automatic checks on GitHub	27
12.3 Accuracy evaluation of the checker	27
12.4 Real results (29 September 2026)	28
12.5 Earlier example on a challenge sample	28
13. Monitoring, accessibility and languages	29
13.1 Monitoring and cost tracking	29
13.2 Accessibility	29
13.3 English and Hungarian interface	30
14. Development log	31
15. Production readiness	33
16. Known limitations and roadmap	34
16.1 Limitations	34
16.2 Roadmap	34
17. Appendix	35
17.1 Glossary	35
17.2 References	35

1. Introduction

1.1 Purpose of CiteFlow

CiteFlow is a web application for university communication teams. It turns a research paper (PDF) into a complete, publish-ready communication package: a press release, tailored posts for LinkedIn, Facebook, Instagram and X, designed post images, carousels, animated posts and a narrated explainer video, in English or Hungarian. Before anything is published, every sentence, including the text printed on images, is checked against the paper, and a person decides what to do with every flagged sentence.

Core principle: AI drafts and verifies. People approve and publish.

1.2 Background and problem

Exaggeration in science news often starts in the institution's own press release. In a BMJ study of university press releases, 40% contained exaggerated advice, 33% exaggerated causal claims and 36% exaggerated inference from animals to humans, and news stories were far more likely to exaggerate when the press release did (Sumner et al., 2014). Generative AI makes writing faster, but without a check it also makes overclaiming faster. Communication teams need both speed and accuracy.

1.3 Context: DEIK.AI Challenge 2026

CiteFlow was built as an entry to the DEIK.AI Challenge 2026 AI Demo Competition at the University of Debrecen, category 2.C, AI-Assisted PR Content Generation. The submission consists of a one-page project description, a demo video of up to three minutes, the live application and the public source code.

1.4 Goals and scope

Goal	How CiteFlow meets it
Speed	A full package (release, 4 posts, image text, video script) is drafted and checked in minutes.
Accuracy	Independent AI fact-check of every sentence, plus deterministic rules for citations and numbers.
Traceability	Every verdict links to the exact passage, shown highlighted on the real page of the paper.
Human control	Nothing is exported before a person reviews the flagged sentences and approves.
Channel-ready output	Platform-specific posts, designed images in three formats, carousels, animations, video.
Safe operation	Accounts, per-user data separation, usage limits, input validation and privacy controls.

Out of scope (for this version): automatic posting through platform APIs (requires app review by Meta, LinkedIn and X), checking claims against literature beyond the uploaded paper, and generating images of real people.

1.5 Intended audience of this document

- **Judges and reviewers:** sections 1–3, 8–9, 12 and 15 (what it does, principles, security, privacy, quality, production readiness).
- **Developers and maintainers:** sections 4–7 and 10–13 (architecture, AI pipeline, data, deployment, tests, monitoring).
- **Communication staff using the app:** section 3 (user guide).

2. Principles and rules

2.1 Product principles

Principle	What it means in practice
AI drafts and verifies, people approve	The model writes and checks; only a person can accept a flagged sentence or approve export.
Source only	All content is generated from the uploaded paper only. No outside facts, numbers, funding or impact claims.
Same strength as the source	No causal claims from correlations, no lab or simulation results presented as real-world, no removed limitations.
No invented quotes	Where a quote belongs, the placeholder [Quote from the researchers to be added after approval] is inserted.
Public text stays natural	Public content never mentions fact-checking, verification or page numbers; the checks are visible only to the team.
Transparency	Stock footage is illustrative; the paper's own figures are used where possible. No images of real people are generated.
Consent	A researcher photo can be added only with the person's consent (stated in the upload field).
Manual publishing	Share buttons prepare the post and open the platform; a person presses Post.
Privacy by design	Minimal data, salted password hashes, per-user folders, deletion on request (section 9).
Fast by default	Parallel AI calls and cached rendering keep waiting time short (section 10).

2.2 Content-generation rules

These rules are part of the generation prompt and are enforced again by the verifier:

- Every factual statement must be supported by the source; never add facts, numbers, dates, funding or impact.
- Keep claim strength equal to the source and keep the key limitation.
- Headline at most 12 words, without hype words such as “breakthrough” or “revolutionary”.
- Press release 300–420 words; LinkedIn 80–140 words with 3 hashtags; Facebook 50–90 words with a question; Instagram 40–80 words with 4–6 hashtags; X at most 260 characters.
- Image text: a kicker, the single most striking number exactly as written in the source, three facts of at most 12 words, and a call to action.
- Video: 5–6 scenes, at most 120 words of narration, a hook first and “what comes next” last; stock-footage queries describe filmable scenes without brands, real people, screens or text.

2.3 Verification rules

The verifier is a separate prompt that did not write the text. It returns one verdict per sentence:

Verdict	Meaning	What happens
SUPPORTED	The source states this with the same strength.	Kept. Rule layer double-checks it.

Verdict	Meaning	What happens
EXAGGERATED	The source supports only a weaker or narrower version (causal overclaim, overgeneralisation, certainty inflation, removed limitation, rounded or widened numbers).	Flagged; faithful rewrite proposed.
UNSUPPORTED	Not in the source, or contradicts it.	Flagged; rewrite or removal proposed.
NOT_A_CLAIM	No checkable fact (hooks, calls to action, hashtags, placeholders).	Kept, not scored.
NEEDS_REVIEW	Set by the rule layer: marked supported but no valid passage cited, or a number not found in the cited passage.	Flagged for a person.
UNCHECKED	The checker returned no verdict for this sentence.	Flagged for a person.
ACCEPTED	A person chose "Keep as is" and takes responsibility.	Treated as approved.

Strictness rules in the verifier prompt: when in doubt between supported and exaggerated, choose exaggerated; words of certainty ("proves") are exaggerated unless the source is equally strong; a result credited to the wrong method or group is exaggerated; rounded or widened numbers ("11–15%" for "11.1–14.7%") are exaggerated.

Deterministic rule layer

After the AI verdict, code checks two rules that do not depend on the model: (1) a SUPPORTED sentence must cite at least one real passage ID; (2) every number in the sentence must appear in the cited passages, after normalising thousand separators, decimal commas and trailing zeros (hashtags are ignored). If either rule fails, the verdict becomes NEEDS_REVIEW with the reason shown.

Public images

Facts judged EXAGGERATED or UNSUPPORTED are never printed on images: the faithful version is used instead, or the fact is left out. Facts that only need review stay, so images are never empty.

3. User guide

3.1 Signing in

Users sign in with a team account (set by the administrator), a self-created account, or the shared demo account (demo@citeflow.app, for reviewers). Creating an account requires a password of at least 8 characters with a letter and a number, and accepting the privacy note.

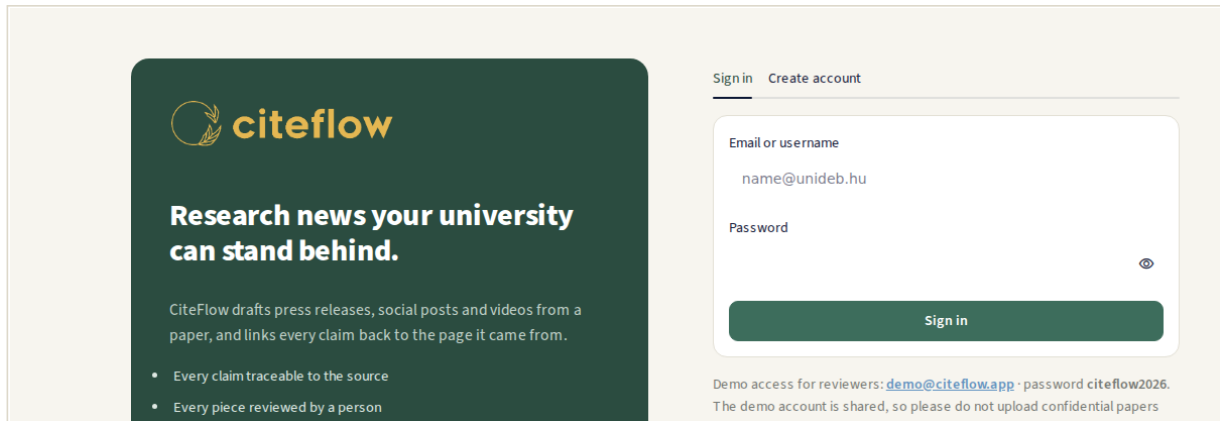


Figure 1 – Sign-in page with the demo-account notice.

3.2 Pages

Page	Purpose
Studio	Upload a paper, choose language and audience, create and review content, publish.
Projects	All saved projects: open, continue or delete.
Insights	AI publishing plan: best days and times per platform, hashtags, format tips, launch-week schedule.
Channels	Save your LinkedIn, Facebook, Instagram, X, blog and YouTube links so posts open in the right place.
Account	Profile, privacy note, sign out, delete account and all data.

3.3 Creating content (Studio)

1. Upload the paper (PDF, up to 25 MB and 80 pages). Optionally add a researcher name and a photo (with consent).
2. Choose the output language (English or Hungarian) and the audience (general public, students, industry partners).
3. Click **Create verified content**. CiteFlow reads the paper, drafts all content and checks every sentence.
4. Review the **needs your decision** panel (below), then the press release, posts, images and video.
5. Use **Refine with feedback** for changes in plain words; everything is checked again.
6. Approve and export the package, or publish with the share buttons.

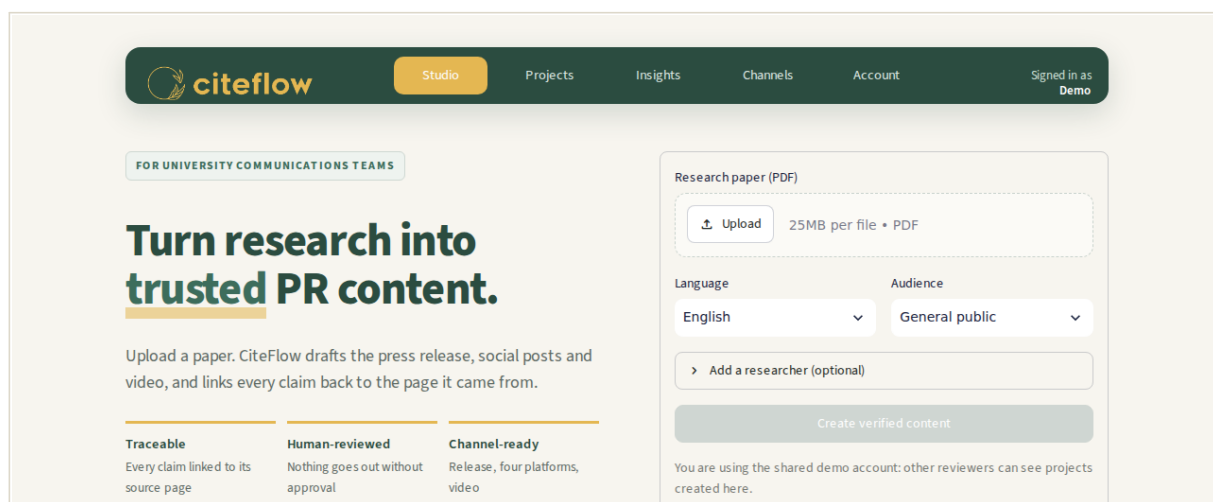


Figure 2 – Studio start screen (demo account notice shown under the button).

3.4 Deciding on flagged sentences

Every flagged sentence across the release, posts, image text and video is listed in one panel with the reason from the paper:

Action	Effect
Use faithful version	Replaces the sentence with the rewrite that matches the paper; re-checked automatically.
Keep as is	Keeps the sentence; the person takes responsibility (verdict ACCEPTED).
Remove	Deletes the sentence from the content.
Fix all	Applies every available faithful version at once.
Keep all remaining	Accepts all remaining flags.
Edit and re-check	Edit a sentence by hand; it is checked again.

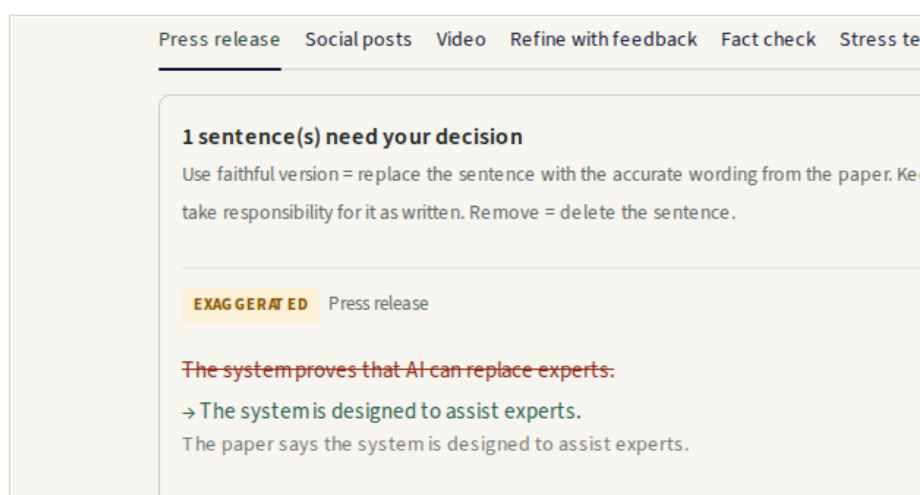


Figure 3 – The decision panel (shown with test content).

3.5 See the proof

Pick any sentence to see the actual page of the paper with the supporting passage highlighted and the sentence's numbers outlined. This lets a reviewer confirm a verdict in seconds.

The screenshot shows the front page of a journal article. At the top, the journal title 'Biomedical Signal Processing and Control' is displayed in a yellow box, along with the Elsevier logo and the journal homepage URL. Below this, the article title 'Automatic screening system for conventional Pap smear analysis' is shown in a yellow box, followed by the authors' names: David Kupas^{a,*}, Ilona Kovacs^b, Andras Hajdu^a, and Balazs Harangi^a. The affiliations are listed below: ^a Department of Data Science and Visualization, Faculty of Informatics, University of Debrecen, Kassai ut 26, Debrecen, 4028, Hajdu-Bihar, Hungary; and ^b Department of Pathology, University of Debrecen Clinical Center, Kossuth Gyula Campus, Bartok Bela ut 2-26, Debrecen, 4031, Hajdu-Bihar, Hungary. The page is divided into two main sections: 'ARTICLE INFO' and 'ABSTRACT'. The 'ARTICLE INFO' section includes keywords: Cervical cancer screening, Deep learning, Ensemble classification, and Pap smear analysis. The 'ABSTRACT' section contains a paragraph of text. A yellow highlight is placed over the first sentence of the abstract. Below the abstract, a note states: 'Highlighted: the passage the checker relied on (page 1, P1-1)'. At the bottom of the page, there is a button labeled 'Edit and re-check' and a footer that reads 'CiteFlow - AI drafts and verifies, people approve - DEIKAI Challenge 2026'.

Biomedical Signal Processing and Control

ELSEVIER

journal homepage: www.elsevier.com/locate/bspc

Automatic screening system for conventional Pap smear analysis

David Kupas^{a,*}, Ilona Kovacs^b, Andras Hajdu^a, Balazs Harangi^a

^a Department of Data Science and Visualization, Faculty of Informatics, University of Debrecen, Kassai ut 26, Debrecen, 4028, Hajdu-Bihar, Hungary
^b Department of Pathology, University of Debrecen Clinical Center, Kossuth Gyula Campus, Bartok Bela ut 2-26, Debrecen, 4031, Hajdu-Bihar, Hungary

ARTICLE INFO

ABSTRACT

Keywords:
Cervical cancer screening
Deep learning
Ensemble classification
Pap smear analysis

This paper presents a comprehensive automatic screening system designed to assist cytological experts in the evaluation of Pap smears for cervical cancer detection. This work presents an end-to-end automatic screening system developed specifically for conventional Pap smears. The goal of the system is to reduce the burden on human experts by providing a reliable, resource-efficient, and accurate solution that can distinguish between normal and diseased smears. The proposed automatic screening system processes whole-slide images by first dividing them into 2000×2000 pixel tiles, then applies a YOLO-based localization algorithm to extract individual cells. These cells are subsequently classified by an ensemble of nine deep neural networks, including convolutional and transformer-based architectures, trained on the publicly available APACC dataset. The final stage involves aggregating cell-level predictions across models to generate smear-level features, which are used to train a LightGBM classifier. The system is evaluated on a private dataset of 339 expert-annotated smears collected in a clinical setting. Despite the relatively small number of smears, over 12 million individual cell predictions were utilized, resulting in an average smear-level classification accuracy of 88.8% using cross-validation. These findings demonstrate the feasibility and potential of such automated systems to support existing cervical cancer screening workflows and to enhance diagnostic consistency and throughput.

Highlighted: the passage the checker relied on (page 1, P1-1).

> Edit and re-check

CiteFlow - AI drafts and verifies, people approve - DEIKAI Challenge 2026

Figure 4 – See the proof: the passage the checker relied on, highlighted on the real page of the paper.

3.6 Other tools

- **Social posts:** one card per platform with a designed image (16:9, 1:1, 4:5), a carousel on demand, a 6-second animated MP4 and a **Publish on ...** button that copies the text, saves the image and opens the platform (X and LinkedIn pre-filled).
- **Video:** renders a 45–60 s narrated video (16:9 or 9:16) from verified lines only, with subtitles (SRT).
- **Stress test:** writes a deliberately hyped release and checks it, to show the checker catches exaggeration.
- **Approve & export:** after confirming the review, download a package with the release, posts, images, video, subtitles and the verification report.

4. Architecture

4.1 Overview

CiteFlow is a single Streamlit application written in Python. The interface (the ui/ package, one module per page), the AI pipeline, media generation and storage are separate modules. All AI calls go to the Google Gemini API; there is no separate backend server. With a PostgreSQL database configured, CiteFlow is physically three-tier: browser, application server and database; without one it runs as a two-tier application with file storage.

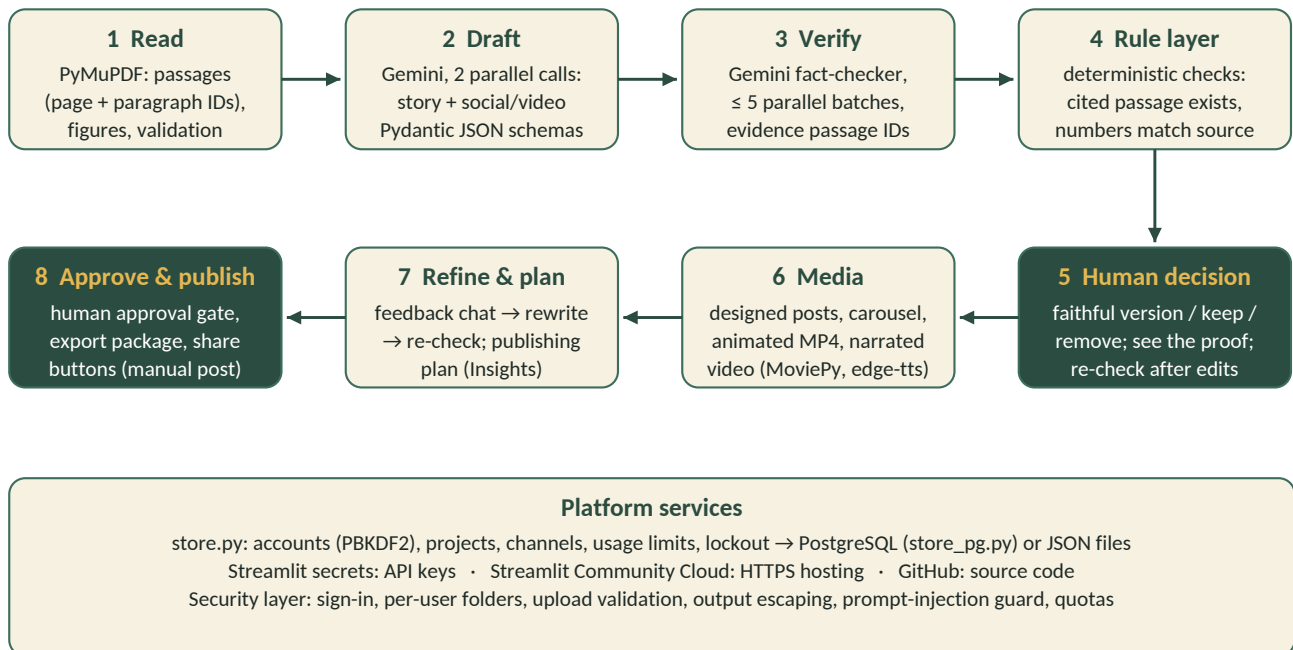


Figure 5 – Processing pipeline and platform services.

4.2 Modules

File	Responsibility
app.py	Entry point: page setup, style, daily retention run, routing between pages.
ui/core.py	Configuration, secrets, usage limits, writer and checker AI clients, monitoring hooks, style, navigation.
ui/project.py	The open project: checking, decisions on flagged sentences, images, saving and reopening.
ui/results.py	Studio results: release, posts, images, video, fact check, feedback, stress test, export.
ui/login.py, ui/page_*.py	Sign-in and one module per page (Studio, Projects, Insights, Channels, Account, Admin).
ui/i18n.py, ui/hu.py	English/Magyar interface switch and the Hungarian texts.
evaluate.py, evaluation/	Accuracy evaluation of the checker on 48 labelled statements.
tests/	52 automated tests with a stand-in AI model; run by GitHub on every upload.
pipeline.py	PDF passages and figures, upload validation, Pydantic schemas, prompts, Gemini client, generation, verification, rule layer, publishing plan.
cards.py	Designed post images, carousels and animated posts in the CiteFlow palette.

File	Responsibility
<code>video.py</code>	Narrated explainer video: stock b-roll, paper figures, word-synced captions, voice-over, SRT.
<code>store.py</code>	Accounts (PBKDF2), sign-in lockout, usage quotas, channels, projects; chooses the storage backend.
<code>store_pg.py</code>	PostgreSQL backend with the same functions (used when DATABASE_URL is set).
<code>migrate_to_postgres.py</code>	Copies existing data/ (accounts, channels, projects, files) into PostgreSQL.
<code>hash_password.py</code>	Command-line helper to create password hashes for team accounts in secrets.
<code>.streamlit/config.toml</code>	Theme, upload limit, XSRF protection, hidden error details.
<code>requirements.txt</code> / <code>packages.txt</code>	Pinned Python packages / system fonts for Streamlit Cloud.
<code>.github/dependabot.yml</code> , <code>.github/workflows/tests.yml</code>	Weekly security-update checks; code checks and tests on every upload.
<code>ruff.toml</code> , <code>pytest.ini</code> , <code>requirements-dev.txt</code>	Code-quality rules and test settings.

4.3 Technology stack

Layer	Technology (version)	Why
Language	Python 3.11+	One language for AI, PDF, images and web UI.
Web interface	Streamlit 1.64.0 (custom CSS, fragments, <code>st.iframe</code>)	Fast to build, free hosting, reactive UI.
AI model	Google Gemini via <code>google-genai 2.25.0</code> ; newest Flash model auto-selected	Strong multilingual quality, JSON output, free tier.
Structured output	<code>Pydantic 2.13.3</code> schemas	Every AI response is validated before use.
PDF	<code>PyMuPDF 1.28.2</code>	Text with coordinates (for highlighting), figures, rendering pages.
Images	<code>Pillow 11.3.0</code> , <code>NumPy 2.4.4</code>	Designed posts, carousels, frames.
Video	<code>MoviePy 2.2.1</code> + FFmpeg	Composition, transitions, H.264 encoding.
Voice	<code>edge-tts 7.2.8</code> (Microsoft neural voices)	Natural English and Hungarian voices with word timings.
Stock footage	Pexels API (optional)	Free, licensed b-roll.
Database	PostgreSQL 16 via <code>psycopg 3.3.6</code> + <code>psycopg-pool 3.3.3</code> (e.g. Neon)	Durable, concurrent, transactional storage.
OCR	Tesseract (English + Hungarian) via <code>PyMuPDF</code>	Reads scanned pages while keeping word positions.
Monitoring	Built-in event log + Admin page; optional Sentry (<code>sentry-sdk 2.71.0</code>)	Usage, cost, errors and crash reports.
Quality	<code>pytest 9.1.1</code> , <code>ruff 0.15.11</code> , GitHub Actions	Automated tests and code checks on every upload.
Hosting	Streamlit Community Cloud, GitHub	Free HTTPS hosting from the repository.

4.4 Session and data flow

Each browser session has its own server-side state (Streamlit session_state): the passages, generated content, verdicts, images and video paths. After every meaningful change the project is saved to disk; a SHA-1 signature of the state prevents unnecessary writes. Heavy sections (proof pages, carousels, animations) run as independent fragments so the rest of the page does not reload.

5. AI pipeline

5.1 Reading the paper

- The upload is validated first: it must start with the PDF signature, open without errors, not be password-protected and have 1–80 pages.
- Text is split into passages of up to about 700 characters, each with an ID of the form P{page} - {n} and its coordinates on the page (used for highlighting).
- Scanned pages (no text layer, but images) are read with Tesseract OCR at 150 dpi. The first scanned page is read in English and Hungarian; if it is not Hungarian, the rest is read in English only, which is about twice as fast (about 5 s instead of 11 s per page). Word positions are kept, so highlighting still works; scanned papers are limited to 30 pages, and the Fact check tab names the pages that were read by OCR.
- Up to 12 figures are extracted and shrunk to at most 1400 px to save memory.
- The prompt receives at most about 150,000 characters of passages, wrapped in explicit START/END markers.

5.2 Generation

Content is generated in two parallel Gemini calls with the same rules: (A) headline, subheadline, institution, card title, press release and image text; (B) the four platform posts, video title and scenes. Both responses must match Pydantic schemas (PartStory, PartSocial) and are merged into one Content object. Temperature 0.7 for natural writing.

5.3 Verification

Every sentence becomes an item with a stable ID. Items are checked in up to five parallel batches of about eight, at temperature 0 and with the model's full reasoning enabled (accuracy over speed).

ID prefix	Content
H1	Headline
R1, R2 ...	Press release sentences
L, F, I, X	LinkedIn, Facebook, Instagram and X post sentences
K0 – K3	Image text: key number (K0) and three facts
V1, V2 ...	Video narration per scene

5.4 Independent checker

By default the fact-checker uses a different Gemini model from the writer (the next-best full Flash model), so the text is not judged by the model that wrote it. The Fact check tab shows which models wrote and checked. The setting CHECKER_MODEL can be *auto* (default), *same* or a specific model name; if the independent model is unavailable, the writer's model is used and the tab says so. The stress test also uses the independent checker.

5.5 Model selection and resilience

- At start-up the app lists the Flash models available to the API key and ranks them newest first (stable before preview, non-lite before lite); up to seven models form a fallback chain.
- Fast mode uses low thinking for generation (Gemini 3.x: thinking level low; 2.5 Flash: thinking budget 0); verification uses full reasoning.
- Temporary errors (503, 429, overload, timeouts) are retried twice per model with back-off, then the next model is tried, for up to about 2.5 minutes. Unavailable models (404) are skipped. Users see a friendly “busy, try again” message instead of an error.

5.6 Refinement, planning and stress test

- **Refine:** the current content and the user's feedback are sent with the source; the model changes only what was asked and the accuracy rules still apply. The result is fully re-checked.
- **Publishing plan:** platform-specific days, times, hashtags, format tips and a launch-week schedule, based on well-established engagement patterns (no live platform data is claimed).
- **Stress test:** a hyped rewrite is generated on purpose and checked with the same verifier.

5.7 Prompt-injection protection

A PDF can contain hidden text such as “ignore previous instructions”. Every prompt therefore states that the source document and content are data, not instructions, and the source is enclosed in explicit markers. Output is still escaped, fact-checked and human-approved, so an injected instruction cannot publish anything by itself.

6. Media generation

6.1 Designed posts

Each post gets a designed image in the CiteFlow palette (deep green, gold, cream): kicker label, title, the key number, three numbered fact boxes and a call to action. Formats: 1200×675 (16:9), 1080×1080 (1:1) and 1080×1350 (4:5). The same layout can be rendered over time, which produces a 6-second animated MP4 in which the elements appear one after another.

6.2 Carousels

For Instagram and LinkedIn: a cover slide, one slide per fact, a figure slide from the paper (if available) and a closing slide; downloadable as a ZIP. Built on demand to keep the page fast.

6.3 Explainer video

- Formats 1280×720 (16:9) and 720×1280 (9:16) at 20 fps, H.264 (ultrafast preset).
- Only verified narration is spoken; flagged lines are replaced by their faithful version.
- Voice-over with Microsoft neural voices (English: Ava, Sonia; Hungarian: Noémi, Tamás) via edge-tts; word-boundary timings drive word-by-word captions. An SRT subtitle file is exported.
- Backgrounds: Pexels stock clips (illustrative) or the paper's own figures; crossfade transitions and slow zoom.
- All voice-over lines and stock clips are fetched in parallel before rendering.

7. Data and storage

7.1 Two interchangeable backends

All storage goes through `store.py`, which offers one set of functions (`create_user`, `check_user`, `save_project`, `load_project`, `use_quota`, ...). A small dispatcher sends each call to one of two backends:

Backend	When it is used	Suited for
PostgreSQL (<code>store_pg.py</code>)	<code>DATABASE_URL</code> is set in Streamlit secrets or the environment	Deployment: data survives restarts; safe under many users
JSON files (<code>data/</code>)	No <code>DATABASE_URL</code>	Local development and quick demos

Because both backends have identical functions, no other part of the app changed when PostgreSQL was added. Tables are created automatically on first start (`CREATE TABLE IF NOT EXISTS`).

7.2 Database schema

Table	Key	Contents
<code>users</code>	<code>username</code>	salt, PBKDF2 hash, iterations, display name, created
<code>channels</code>	<code>username</code>	saved channel links (JSONB)
<code>projects</code>	<code>username + pid</code>	meta and full state as JSONB, last update; indexed by user and date
<code>project_files</code>	<code>username + pid + name</code>	uploaded paper and rendered video/subtitles (BYTEA); deleted with the project (<code>ON DELETE CASCADE</code>)
<code>login_failures</code>	<code>key (account or ip:...)</code>	time of each failed sign-in in the last 10 minutes
<code>usage</code>	<code>day + key</code>	today's counters for runs, checks and sign-ups

7.3 How files are handled

The app writes rendered videos to a local working folder. When a project is saved, `store_pg.py` uploads every new or changed file in that folder to `project_files`; when a project is opened, missing files are written back to the folder. The local folder is therefore only a cache and may be wiped at any time (for example when Streamlit Cloud restarts) without losing data.

7.4 Consistency and concurrency

- A connection pool (1–6 connections) is shared by all sessions; broken connections are detected and replaced, so the app recovers when a serverless database wakes up after sleeping.
- Usage limits are checked and counted in one transaction with row locks (`SELECT ... FOR UPDATE`): in a test with 25 simultaneous requests and a limit of 15, exactly 15 were allowed.
- In file mode, writes use a temporary file plus atomic rename, and a process-wide lock serialises read-modify-write operations.
- A database error while saving is reported to the app as a normal save error, so a full database never crashes the page.

7.5 Hosted database and limits

Any PostgreSQL service works. For the demo, a free Neon project is recommended because its database sleeps after 5 minutes idle and wakes automatically on the next request; free Supabase projects pause after a week of inactivity and must be restored by hand. Both free tiers allow about 0.5 GB, enough for dozens of projects with videos. In production, large files (papers, videos) should move to object storage with only their references in the database (see Roadmap).

```
store.py -> DATABASE_URL set?
          yes -> store_pg.py -> PostgreSQL (users, channels, projects, files, usage)
          no  -> JSON files under data/
```

8. Security

8.1 Approach

Security follows the OWASP Top 10 categories and the principle of least privilege. On 27 September 2026 a code review of app.py, store.py, the configuration and the repository produced ten findings; all were addressed in version 1.1 and verified by automated tests.

8.2 Threat model

Asset	Threat	Controls
Gemini API quota / cost	Automated or repeated use drains the quota; judges see “busy” errors.	Per-user, demo and global daily limits; sign-up limit per network; key restricted in Google Cloud.
User accounts	Password guessing, credential stuffing, stolen password files.	PBKDF2-SHA256 600,000 iterations with salt, lockout, password policy, no plain text.
Users' papers and projects	Access by other users.	Per-user folders, sanitised folder names, server-side project IDs, sign-in required.
Application availability	Huge or malformed uploads exhausting memory.	25 MB upload limit, PDF validation, 80-page limit, figure thumbnails.
Browsers of users	Cross-site scripting through AI or user text.	All dynamic text HTML-escaped; share data JSON-encoded; only http(s) links.
Content integrity	Prompt injection in a PDF steering the output.	Injection guard in every prompt, source delimiters, verification and human approval.
Secrets	API keys leaked through code or Git.	Keys only in Streamlit secrets; .gitignore; repository checked.

8.3 Controls implemented

Authentication

- Passwords are stored as PBKDF2-HMAC-SHA256 hashes with a random 16-byte salt and 600,000 iterations (OWASP 2023). Older hashes (120,000 iterations) are upgraded automatically at the next successful sign-in.
- Comparisons are constant-time; a sign-in for an unknown account does the same hashing work, so response time does not reveal which accounts exist.
- Password policy: 8–128 characters, at least one letter and one number, not a common password and not equal to the username.
- Team accounts in secrets can be stored as hashes (pbkdf2_sha256\$...) created with hash_password.py.

Brute-force protection

- After 5 failed sign-ins for an account within 10 minutes, that account is locked for the rest of the window, even for the correct password.
- After 20 failed sign-ins from one network address within 10 minutes, that address is blocked for the rest of the window.
- Every failed attempt waits 0.6 s and shows the same generic message.

Authorisation and data separation

- Every page except sign-in requires an authenticated session.

- Each user's data lives in a separate folder; usernames are validated (3–40 characters from a safe set) and sanitised before use in paths, and project IDs are generated by the server, which prevents path traversal.
- The shared demo account cannot delete projects or its account, and shows a notice not to upload confidential papers.

Input validation

- Upload limit 25 MB (server setting); PDF signature, integrity, encryption and page-count checks before processing.
- Maximum lengths on all text fields (name 80, username 40, password 128, feedback 1000, researcher 100 characters).
- Channel links are used only if they start with http(s); anything else falls back to the platform's home page.

Output encoding

All text from users, papers or the AI that is placed into HTML passes through `html.escape`. Data for the publish buttons is JSON-encoded, with “</” escaped, inside a sandboxed iframe. Hidden error details (`showErrorDetails = false`) prevent stack traces reaching visitors.

Abuse and quota limits

Limit (per day)	Default	Applies to
<code>user_daily_runs</code>	15	Generate, rework, stress test, publishing plan, video render (per user)
<code>demo_daily_runs</code>	60	The shared demo account
<code>global_daily_runs</code>	300	All users together (protects the shared quota)
<code>user_daily_checks</code> / <code>demo_daily_checks</code>	80 / 250	Re-checks after fixing or editing sentences
<code>signups_per_ip_per_day</code>	3	New accounts from one network
<code>max_pdf_pages</code>	80	Pages per uploaded paper

Limits can be changed without code under `[limits]` in secrets. A blocked action is not counted and shows a clear message.

Database security

- All queries use bound parameters (psycpg placeholders), never string formatting of user input, which prevents SQL injection.
- The connection string (including its password) is a secret: stored only in Streamlit secrets, never in the repository.
- Hosted databases require TLS (`sslmode=require` in the connection string), so data is encrypted between the app and the database.
- Every query is scoped to the signed-in username; deleting an account removes its projects and files through cascading deletes.

Transport, secrets and dependencies

- HTTPS is provided by Streamlit Cloud; XSRF protection is enabled.
- API keys exist only in Streamlit secrets (never in code, never shown to users). `secrets.toml` and `data/` are excluded by `.gitignore`; on 27 Sep 2026 the public repository was checked and contains neither.
- All Python packages are pinned to exact, tested versions; Dependabot checks weekly for security updates.

8.4 Review findings and status

#	Finding (27 Sep 2026)	Severity	Resolution in v1.1
1	No usage limits: open sign-up could drain the shared AI quota	High	Daily per-user, demo and global limits; sign-up limit per IP
2	Shared public demo account could delete others' projects	Medium	Demo cannot delete; notice about shared data
3	Weak brute-force protection (0.6 s delay only)	Medium	Lockout per account (5) and per IP (20) for 10 min
4	No upload size or page limit (memory exhaustion)	Medium	25 MB limit, PDF validation, 80-page limit
5	Weak password policy (6 chars), PBKDF2 at 120k	Low-Med	8+ chars, letter+number, common list; 600k with upgrade
6	Team passwords in secrets as plain text	Low	Hash format supported; hash_password.py helper
7	Prompt injection through PDF text	Low	Guard in all prompts, source delimiters
8	No privacy notice or account deletion (GDPR)	Low (demo)	Privacy note + consent; delete account and all data
9	Dependencies not pinned	Low	Exact pins; Dependabot
10	Possible secrets in the public repository	Check	Verified: secrets.toml and data/ not in the repository

8.5 Residual risks

- The Gemini free tier is shared: even with limits, heavy parallel use by several reviewers can briefly hit Google's rate limit.
- Network-address limits depend on the address Streamlit reports; users behind one network share a limit, and a determined attacker can change address.
- Without a database, files on Streamlit Cloud reset on restart; with the free database tier, storage is limited (about 0.5 GB). Confidential papers should not be uploaded to the public demo account.
- The verifier is an AI model and can make mistakes; human approval remains mandatory.
- The Git history could not be inspected automatically; if a key was ever committed, it must be rotated (section 11.7).

9. Privacy and responsible AI

9.1 Personal data processed

Data	Purpose	Where
Name, email or username	Account and sign-in	data/users.json
Password	Sign-in (only a salted hash is stored)	data/users.json
Uploaded papers and generated content	Provide the service, reopen projects	User's project folder
Researcher name and photo (optional)	Video and posts, only with consent	Session; video file
Channel links	Address posts and open the right page	User's channels.json
Network address (IP)	Abuse protection (sign-in and sign-up limits)	security/*.json, kept up to one day

9.2 Third-party processors

- **Google Gemini API:** paper text and content for generation and checking.
- **Microsoft Edge text-to-speech:** video narration text only.
- **Pexels:** short English keywords for stock footage only.
- **Streamlit Community Cloud / GitHub:** hosting and source code.

9.3 User rights and controls

- A privacy note is shown at sign-up and on the Account page; creating an account requires accepting it.
- Users can delete any project, or their whole account with all files (password confirmation required).
- Retention: projects not changed for 365 days (14 days for the shared demo account) are deleted automatically, once a day; both periods are settings, and the privacy note states the values in force. Monitoring events are kept for 90 days.
- Nothing is sold or used for advertising. Data minimisation: no tracking, no analytics (Streamlit usage statistics disabled).

CiteFlow is a competition prototype. Before real deployment, the University's data-protection officer should review the privacy note and data-processing agreements under the GDPR. This documentation is not legal advice.

9.4 Responsible AI

- Human oversight: every flagged sentence needs a human decision; export requires explicit confirmation.
- Transparency: every verdict is explained and linked to the source page; limitations are stated in the app and README.
- No synthetic people: no generated faces or voices imitating real people; real photos only with consent.
- Honest scope: the checker compares with the uploaded paper only, not the wider literature; publishing plans are guidance, not live analytics.

10. Performance

Technique	Effect
Two parallel generation calls	Story and social/video parts written at the same time.
Up to 5 parallel verification batches	Checking time roughly divided by the number of batches.
Low thinking for drafting, full for checking	Fast writing; careful verification.
Fragments for heavy sections	Proof pages, carousels and animations update without reloading the page.
Caching	Post images as PNG/JPEG bytes; proof pages at zoom 1.6, at most 12 kept; model list cached.
Save only on change	SHA-1 signature of the project state avoids repeated disk writes.
Video: 20 fps, static backgrounds, crop-then-resize, parallel voice and footage	Several times faster rendering.
Figure thumbnails (1400 px) and small proof cache	Keeps memory within Streamlit Cloud limits.

11. Deployment and operations

11.1 Run locally

```
pip install -r requirements.txt
copy .streamlit\secrets.toml.example .streamlit\secrets.toml    (then paste your keys)
streamlit run app.py
```

11.2 Configuration (secrets)

```
GEMINI_API_KEY = "... "          # required
PEXELS_API_KEY = "... "          # optional, stock footage

[users]                           # optional team accounts; demo line keeps the demo account
"demo@citeflow.app" = "citeflow2026"
"anna.kovacs@unideb.hu" = "pbkdf2_sha256$6000000$<salt>$<hash>"    # from: python hash_password.py

[limits]                           # optional, defaults shown in section 8
user_daily_runs = 15
```

11.3 Publish updates to GitHub

1. Open github.com/tasvebyousaf/citeflow, choose **Add file → Upload files**.
2. Drag the changed files (and folders such as .github or docs) from the local project folder. Never upload secrets.toml or data/.
3. Click **Commit changes**. Streamlit Cloud redeploys automatically within about a minute.

11.4 Streamlit Community Cloud

- App URL: citeflow-ai.streamlit.app; main file app.py; secrets pasted under **Settings → Secrets**.
- Free apps sleep after a period without visitors and show “This app has gone to sleep”; anyone can wake it with the button (about 30–60 s).
- Logs: **Manage app** (bottom right) shows the build and run log; after an update it prints “Updated app!”.

11.5 Connecting a PostgreSQL database

1. Create a free project at neon.com (choose a Europe region if offered) and copy its connection string (postgresql://...?sslmode=require).
2. Add it to secrets: locally in .streamlit/secrets.toml and in Streamlit Cloud under **Settings → Secrets** as DATABASE_URL = "postgresql://...".
3. Optional: copy existing local data with python migrate_to_postgres.py.
4. Restart the app. Tables are created automatically. To verify: create an account, reboot the app from **Manage app**, and sign in again.

11.6 Creating team accounts

Run python hash_password.py in the project folder, enter the email and password, and paste the printed line under [users] in secrets (locally and in Streamlit Cloud).

11.7 Incident response: leaked API key

1. Create a new key in Google AI Studio and delete the old one immediately.
2. Update the key in Streamlit Cloud secrets and the local secrets.toml.
3. If the key was committed to GitHub, deleting the file is not enough: the old key stays in the history, so rotation is the fix.

12. Testing and quality assurance

12.1 Automated tests

The tests live in the repository (tests/) and run with `pytest`. A stand-in AI model (tests/fake_llm.py) returns fixed content and judges sentences with simple rules, and a small synthetic paper is generated for each run, so no API key, quota or copyrighted PDF is needed. Every test gets its own empty storage; PostgreSQL is used only when `TEST_DATABASE_URL` is set, never the real database from secrets.

File	Covers
<code>test_pipeline.py</code>	Passages and IDs, upload validation, OCR of scanned pages, number rule, prompt guard, verification, image text
<code>test_store.py</code>	Accounts, password policy and hashes, lockout, quotas (also 20 simultaneous requests), projects, isolation, retention
<code>test_app.py</code>	Sign-in, sign-up, demo restrictions, decision panel, reopening projects, feedback, Insights, limits, independent checker
<code>test_monitoring.py</code>	AI calls and errors are recorded; the Admin page is visible to admins only and shows usage and cost
<code>test_evaluation.py</code>	Evaluation metrics and report; checker model selection
<code>test_i18n.py</code>	Every interface text has a Hungarian version with the same placeholders and markup; the app works in Magyar
<code>test_media.py</code>	Post image formats, animation frames, carousel slides

12.2 Automatic checks on GitHub

A GitHub Actions workflow runs on every upload: it installs the pinned packages and Tesseract, runs the ruff code checks, then the full test suite twice, with file storage and with a real PostgreSQL 16 database. A green tick next to an upload means all checks passed. On 29 September 2026 all 53 tests passed in both modes, also in a clean installation of the exact package versions.

12.3 Accuracy evaluation of the checker

evaluation/claims.json contains 48 statements about four sample papers (Pap smear screening, PV forecast post-processing, UAV obstacle avoidance, observatory placement): 23 faithful and 25 distorted in ten ways: changed or rounded numbers, overclaims, certainty inflation, overgeneralisation, wrong attribution, simulation presented as real, removed limitation, contradiction and invented facts. Every faithful statement was checked against the paper text, and every invented fact was confirmed absent from the paper. `evaluate.py` sends them through exactly the same checking code as the app and reports recall (distortions caught), precision (flags that were real problems), false-alarm rate, results per kind, and every miss and false alarm.

A dry run with the stand-in model (which approves everything) isolates the deterministic rule layer: on the four real papers it caught all 7 number and rounding distortions (including “11–15%” for “11.1–14.7%”) and raised no false alarm on the 23 faithful statements. The real checker's figures come from running `evaluate.py` with a Gemini key; reports are saved in `evaluation/results/`.

12.4 Real results (29 September 2026)

The evaluation was run twice with a Gemini key: once with the default setting (auto, an independent checker) and once with the writer model checking itself (same).

Measure	Run 1 (auto)	Run 2 (same)
Distortions caught (recall)	92% – 23 of 25	88% – 22 of 25
Flags that were real problems (precision)	100%	100%
False alarms on faithful statements	0 of 23	0 of 23
Missed	A12, B22	A12, B22, D46

Caught in both runs: changed numbers (5/5), rounded numbers (2/2), certainty inflation (4/4), invented facts (4/4), overclaims, simulation presented as real, removed limitation and contradiction.

What the misses teach: A12 presents a sensitivity of 96.6% as specificity. It passed the number rule because 96.6% does appear in the paper, only as a different measure; the rule checks that numbers exist in the cited passage, not what they measure. B22 describes a Hungarian study as “across Europe”, a subtle overgeneralisation (the paper mentions a European forecasting centre). D46, missed only in run 2, is the same kind: a method “demonstrated on meteor camera networks across Europe”. Overgeneralisation is the weakest area. A12 points to a future rule that compares the measure named next to each number.

Why the two runs cannot be compared. Google’s servers were busy during both runs. In run 2 the intended model (gemini-3.8-flash) was refused 16 times, and 7 of the 8 batches were checked by the fallback gemini-3.5-flash-lite, 1 by gemini-3.7-flash. Run 1 was most likely similar. So the difference between 92% and 88% says nothing about whether an independent checker is better; it is within normal run-to-run variation of one small model. The fair summary is: **22–23 of 25 distortions caught (88–92%) across two runs, with no false alarms.** The set has 48 statements and was written by the developer, so the figures are indicative, not a benchmark. Both runs should be repeated at a quiet time, when the intended models answer.

Lesson for the tool: when every model was busy, the first version of evaluate.py crashed. It now waits 30 and then 60 seconds and tries the paper again; if the service is still busy it skips that paper and marks the report as an incomplete run, so a partial result can never look like a full one. The report also lists which models actually did the checking.

12.5 Earlier example on a challenge sample

On the sample photovoltaic-forecasting paper (Hungarian output), an earlier version checked 29 sentences: 23 supported, 4 non-claims and 2 flagged. Both flagged sentences were social posts that had rounded the paper’s “11.1–14.7%” to “11–15%”.

13. Monitoring, accessibility and languages

13.1 Monitoring and cost tracking

- Every AI call is recorded: user, model, purpose, tokens in, out and for reasoning, duration, and success or error (retries and failed attempts included). App errors are recorded where they are caught (creating, reworking, re-checking, planning, video).
- Anything that looks like an API key is removed from error messages before they are stored. Events are kept for 90 days.
- The Admin page (only for users listed in ADMINS) shows AI calls, failed calls, tokens, estimated cost and active users for 1–90 days, a daily chart, tables by model and user, and the latest errors. Costs appear when prices per 1M tokens are set under [pricing].
- Optional crash reporting to Sentry with SENTRY_DSN (no personal data sent).

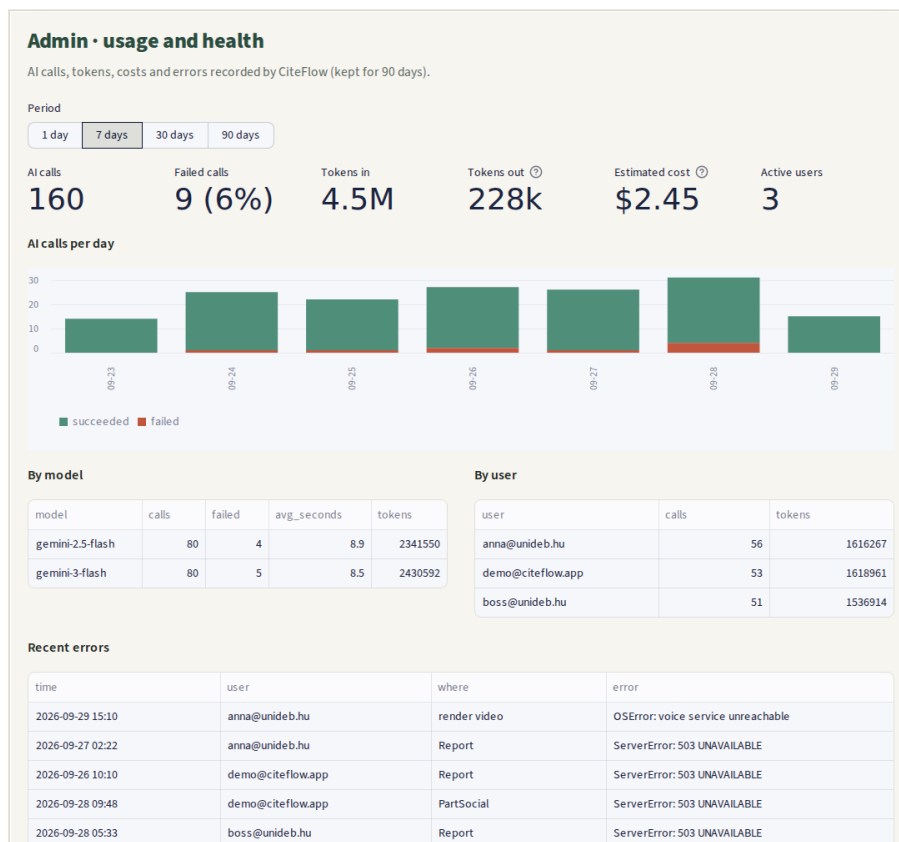


Figure 6 – Admin page (sample data).

13.2 Accessibility

- Text contrast meets WCAG AA (at least 4.5:1) for all text colours; two failing pairs were fixed.
- Verdicts are shown by line pattern as well as colour (solid, double, thick, dashed, dotted), and the legend shows the patterns.
- Screen readers hear the verdict after each flagged sentence; logos have text alternatives; keyboard focus is clearly outlined.
- Content in Hungarian is marked with lang="hu" so screen readers pronounce it correctly; tooltips have a keyboard alternative (See the proof).

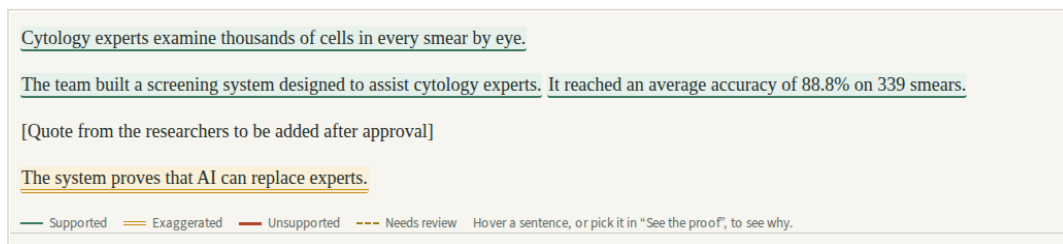


Figure 7 – Verdicts shown by pattern and colour.

13.3 English and Hungarian interface

The interface can be switched between English and Magyar on the sign-in and Account pages; browsers set to Hungarian start in Magyar. Only displayed text is translated, internal values stay in English, so the logic is identical in both languages. The Hungarian texts (about 250) live in ui/hu.py; a test fails if a new interface text has no Hungarian version or if a translation changes a placeholder. The texts should be reviewed by a native speaker. The content language (release, posts, video) is chosen separately in the Studio.

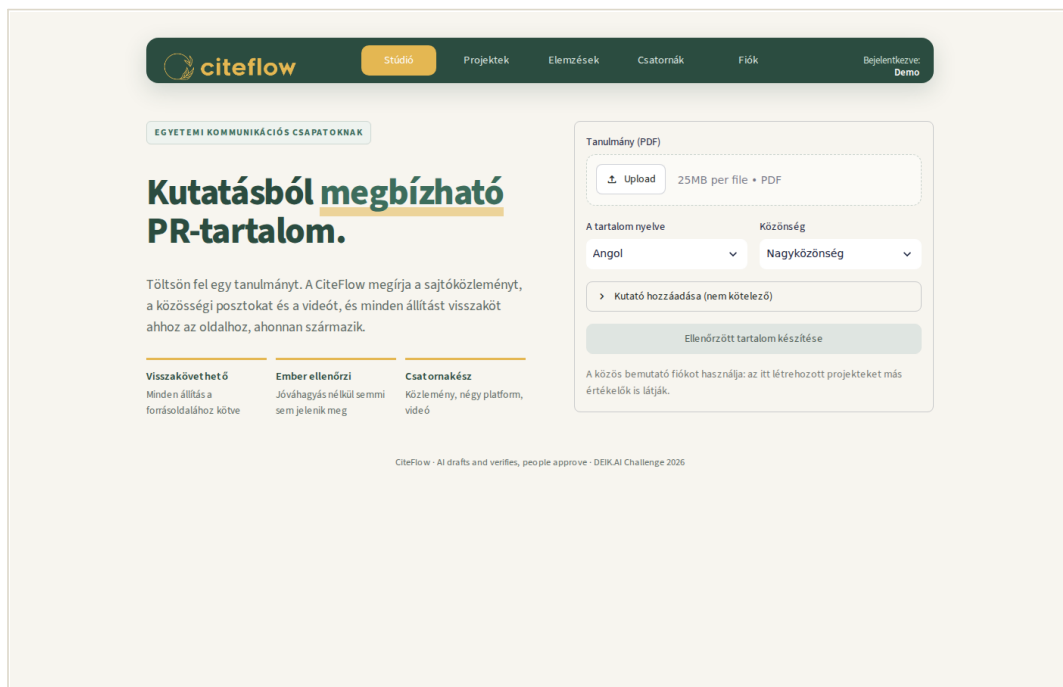


Figure 8 – The Studio in Magyar.

14. Development log

A chronological record of how CiteFlow was built and every significant decision. Entries before 27 September 2026 were reconstructed from the project history.

Date	Step	What was done and why
25 Sep 2026	Idea selection	Reviewed the competition brief and earlier entries. Environmental ideas (such as ozone and air-quality monitoring) were explored and rejected as too similar to an existing entry (GreenMind). Learning from the previous winner's focus on verified reasoning, category 2.C was chosen with verification as the differentiator.
25 Sep 2026	First build (VeriPress)	Streamlit app with PDF passages, Gemini generation, a separate verifier, highlighted proof pages, deterministic number rule, stress test, human approval gate and a first video generator.
25 Sep 2026	Model errors fixed	404 (model not available to new keys) → automatic selection of the newest available Flash model. 503 (busy) → retries, model fallback and a friendly message. Invalid-key errors → secrets values cleaned of spaces, quotes and placeholders.
25 Sep 2026	Video redesign	Slide-style video replaced by stock b-roll, the paper's own figures, word-synced karaoke captions and smooth transitions, in 16:9 and 9:16.
25 Sep 2026	CiteFlow brand and deployment	Renamed to CiteFlow with the tagline "Turn research into trusted PR content"; logo extracted in gold and green; palette applied. Code published on GitHub and deployed to Streamlit Community Cloud (citeflow-ai).
25 Sep 2026	Workspace features	Top navigation; sign-in, sign-up and connected channels; publish buttons that open each platform; public content without any mention of verification; feedback chat; Insights publishing plan; Projects page.
25 Sep 2026	Designed posts and decisions	cards.py: designed images in three formats, carousels and animated posts. "Needs your decision" workflow with faithful version, keep and remove. Fix for empty images: only wrong facts are removed.
25 Sep 2026	Robustness	Tolerant loading of older projects (ValidationError fix); removed an unsupported fragment rerun; replaced deprecated components.html with st.iframe.
25 Sep 2026	Speed	Parallel generation and batched parallel verification, caching, fragments, faster video rendering and parallel voice-over and footage downloads.
25 Sep 2026	Memory	Figure thumbnails and a small proof-image cache to stop out-of-memory crashes on Streamlit Cloud.
25 Sep 2026	Submission material	One-page project description, word-for-word video script, demo checklist, animated end slide and an A4 project overview with the full tech stack.
27 Sep 2026	Version sync	Found that the last performance and memory fixes (app.py, pipeline.py, video.py) had not reached the local folder or GitHub; synced the latest files.
27 Sep 2026	Security review	Code review against OWASP categories: 10 findings (section 8.4).
27 Sep 2026	Security hardening (v1.1)	Usage limits, demo protections, sign-in lockout, upload validation, stronger password hashing and policy, hashed team passwords, prompt-injection guard, privacy note with consent, account deletion, pinned dependencies, Dependabot. 22 new automated security checks; all existing tests pass.
27 Sep 2026	Documentation	This document created (v1.1) as a living record, kept in docs/ in the repository.
28 Sep 2026	Architecture review	Reviewed hosting options, production readiness, AI stack (long-context grounding instead of RAG; API instead of MCP) and shortcomings; wrote an interview guide and contacted the university communications office.

Date	Step	What was done and why
29 Sep 2026	PostgreSQL (v1.2)	Added a PostgreSQL backend behind the unchanged storage functions, chosen by DATABASE_URL, with the JSON files as fallback. Papers and videos are stored in the database and cached locally, so data survives restarts. Migration script; 44 new database tests; all existing tests pass in both modes.
29 Sep 2026	Production readiness (v1.3)	Items 1, 3, 7, 9, 11, 12 and part of 6 of the production review: test suite in the repository with GitHub checks; 48-statement accuracy evaluation; independent checker model; monitoring with Admin page and optional Sentry; OCR for scanned PDFs (table parsing tested and rejected); automatic data retention; app.py split into a ui/ package with ruff checks; Hungarian interface; accessibility pass. Found and fixed during the work: a language switch that could revert the language, and unsorted error lists on the Admin page. 52 tests pass with files and with PostgreSQL.
29 Sep 2026	First accuracy result	Real evaluation run: 23 of 25 distortions caught (92%), no false alarms, precision 100%. The report now lists every model that checked (busy models fall back), and a harmless library warning was silenced.
29 Sep 2026	Second accuracy run (v1.4)	Run with the writer checking itself: 22 of 25 (88%), no false alarms. Both runs were mostly checked by a fallback model because Google's servers were busy, so they cannot compare the two settings; reported as 88–92%. evaluate.py crashed when every model was busy; it now retries after 30 and 60 seconds and skips a paper with an 'incomplete run' note instead. 53 tests pass.

15. Production readiness

On 28 September 2026 CiteFlow was reviewed as a production system. The table shows each shortcoming and its status; items marked *production* were deliberately deferred until a real deployment because they need organisational decisions or paid services.

#	Shortcoming	Status
1	Checker accuracy not measured	Done: 48-statement evaluation set and evaluate.py; 88–92% caught, no false alarms (section 12)
2	Model can change by itself	Production: pin the model version; upgrade only after the evaluation passes
3	Same model writes and checks	Done: independent checker model by default
4	No audit trail of decisions	Production: append-only decision log in the database
5	No roles or approval workflow	Production: editor/approver/admin roles via university SSO; researcher approval link
6	Hosting, storage, data protection	Partly done: PostgreSQL in Frankfurt (EU), retention rule. Production: paid Gemini tier, EU app hosting, data-processing agreement
7	Tests not in the repository	Done: tests/ and GitHub checks on every upload
8	Heavy work blocks the server	Production: background job queue for video rendering
9	No monitoring or cost tracking	Done: event log, Admin page, optional Sentry
10	Copyright of paper figures	Production: detect or ask for the paper's licence before reusing figures
11	Scanned PDFs and tables	Done: OCR. Structured table parsing tested and rejected (up to 12.5 s extra per paper, missed tables); table numbers are already captured as text
12	Code and interface maturity	Done: ui/ modules, ruff checks, Hungarian interface, accessibility pass

16. Known limitations and roadmap

16.1 Limitations

- The verifier is an LLM: a strong second pair of eyes, not a replacement for the researchers' approval.
- Claims are checked against the uploaded document only.
- Stock footage is illustrative, not footage of the research.
- Posting is manual; platform APIs require app review.
- Very long documents are truncated to about 150,000 characters; papers are limited to 80 pages.
- Without a database, storage on the free hosting resets on restart; the free database tier holds about 0.5 GB.
- OCR takes about 5–11 seconds per page and can misread characters; scanned papers are limited to 30 pages.
- Streamlit's own controls (for example the file uploader's “Upload” text) stay in English in the Hungarian interface.
- The AI-written publishing plan is in English; the Hungarian interface texts need a native speaker's review.

16.2 Roadmap

Priority	Item
High	Object storage for papers and videos (only references in PostgreSQL), automated backups, encryption at rest.
High	University single sign-on (Microsoft Entra ID / OpenID Connect via Streamlit st.login) instead of local passwords.
Medium	Direct publishing through the LinkedIn, Meta and X APIs after app review, still with human approval.
Medium	Researcher approval link: the researchers confirm quotes and flagged claims before release.
High	Deferred production items 2, 4, 5, 8 and 10 from section 15 (pinned model, audit log, roles, job queue, figure licences).
Low	More languages and voices; brand-kit upload for other institutions.

17. Appendix

17.1 Glossary

Term	Meaning
Passage	A numbered piece of the paper's text (P{page}-{n}) used as evidence.
Verdict	The checker's judgement of a sentence (section 2.3).
Faithful version	A rewrite proposed by the checker that matches the paper's strength.
Rule layer	Deterministic checks after the AI verdict: cited passage exists, numbers match.
Run	One expensive AI action (generate, rework, stress test, plan, video) counted against the daily limit.
PBKDF2	Password-based key derivation function; makes stored password hashes slow to crack.
Prompt injection	Text inside input data that tries to give the AI new instructions.
XSS	Cross-site scripting: injecting code into web pages viewed by others.

17.2 References

- Sumner, P. et al. (2014). The association between exaggeration in health related science news and academic press releases: retrospective observational study. BMJ 349:g7015.
- OWASP Top 10 (2021) and OWASP Password Storage Cheat Sheet (PBKDF2-HMAC-SHA256: 600,000 iterations).
- Streamlit documentation: secrets management, configuration options, Community Cloud.
- Google Gemini API documentation: structured output and thinking configuration.